

Lab-10 INTRODUCTION TO BISON

- Name - Aditya Sinha
- Reg.No - 230905218
- CSE-A-27

Code (q1.l)

```
%{
#include "q1.tab.h"
%}

%%
", " return COMMA;
int      { return INT; }
float    { return FLOAT; }
char     { return CHAR; }
[a-zA-Z_][a-zA-Z0-9_]* { return ID; }
;        { return SEMI; }
[ \t\r]+ ;
\n       ;
.        { return 0; }
%%
```

Code (q1.y)

```
%{
#include <stdio.h>
#include <stdlib.h>
%}

%token INT FLOAT CHAR ID SEMI COMMA

%%
start: decl SEMI { printf("Valid declaration\n"); exit(0); }
      ;

decl: type id_list
     ;

id_list: ID
        | ID COMMA id_list
        ;

type: INT | FLOAT | CHAR;
%%

int yyerror(char *msg) {
```

```

        printf("Invalid declaration\n");
        exit(1);
    }

    int main() {
        printf("Enter declaration: ");
        yyparse();
        return 0;
    }

```

Output

```

CD_A1@CL3-25:~/Desktop/College/Sem6/CD/Lab/Lab10$ ./q1
Enter declaration: int a, n,g;
Valid declaration

CD_A1@CL3-25:~/Desktop/College/Sem6/CD/Lab/Lab10$ ./q1
Enter declaration: int a =5;
Invalid declaration

```

Code (q2.l)

```

%{
#include "q2.tab.h"
#include <stdlib.h>
%}

%%

"if"           { return IF; }
"else"         { return ELSE; }

"=="|"!="| "<="| ">="| "<"| ">"    { return RELOP; }

"="            { return ASSIGN; }
";"           { return SEMI; }

"("            { return LPAREN; }
")"           { return RPAREN; }

[0-9]+         { yylval.num = atoi(yytext); return NUMBER; }

[a-zA-Z_][a-zA-Z0-9_]* { return ID; }

[ \t\r]+      ;
\n            ;

.              { return yytext[0]; }

```

%%

Code (q2.y)

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(const char *s);
%}

%union {
    int num;
}

%token IF ELSE ID RELOP ASSIGN SEMI LPAREN RPAREN
%token <num> NUMBER

%left '+' '-'
%left '*' '/'
%nonassoc IFX
%nonassoc ELSE

%%

start: ifstmt SEMI
    {
        printf("Valid decision statement\n");
        exit(0);
    }
    ;

ifstmt:
    IF LPAREN cond RPAREN assign_stmt %prec IFX
    | IF LPAREN cond RPAREN assign_stmt ELSE assign_stmt
    ;

cond: expr RELOP expr ;

expr: expr '+' term
    | expr '-' term
    | term
    ;

term: term '*' factor
    | term '/' factor
    | factor
    ;
```

```

factor:
    ID
    | NUMBER
    ;

assign_stmt: ID ASSIGN expr ;

%%

void yyerror(const char *s) {
    printf("Invalid decision statement\n");
    exit(1);
}

int main() {
    printf("Enter decision statement: ");
    yyparse();
    return 0;
}

```

Output

```

CD_A1@CL3-25:~/Desktop/College/Sem6/CD/Lab/Lab10$ ./q2
Enter decision statement: if(x+y>5) z = 5;
Valid decision statement

```

Code (q3.l)

```

%{
#include "q3.tab.h"
%}

%%
[0-9]+      { yylval = atoi(yytext); return NUMBER; }
\+         { return '+'; }
\-         { return '-'; }
\*         { return '*'; }
\/         { return '/'; }
\n         { return NL; }
[ \t\r]+   ;
.          { return 0; }
%%

```

Code (q3.y)

```

/*
Bison program to evaluate arithmetic expressions with +, -, *, /
*/

%{
#include <stdio.h>
#include <stdlib.h>
%}

%token NUMBER NL
%left '+' '-'
%left '*' '/'

%%
stmt: expr NL { printf("Valid Expression\n"); exit(0); }
;
expr: expr '+' expr
    | expr '-' expr
    | expr '*' expr
    | expr '/' expr
    | NUMBER
;
%%

int yyerror(char *msg) {
    printf("Invalid Expression\n");
    exit(1);
}

int main() {
    printf("Enter arithmetic expression: ");
    yyparse();
    return 0;
}

```

Output

```

CD_A1@CL3-25:~/Desktop/College/Sem6/CD/Lab/Lab10$ ./q3
Enter arithmetic expression: 3/5+6*2
Valid Expression

```

Code (q4.l)

```

%{
#include "q4.tab.h"
#include <stdlib.h>
%}

```

```

%%

[0-9]+      { yylval = atoi(yytext); return NUM; }

\+          { return '+'; }
\-          { return '-'; }
\[          { return '*'; }
\[          { return '/'; }
\[          { return '^'; }

\n          { return NL; }

[ \t\r]+    ;

.           { return yytext[0]; }

%%

```

Code (q4.y)

```

%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(const char *s);
%}

%token NUM NL

%%

input:
    exp NL
    {
        printf("Valid postfix expression\n");
        exit(0);
    }
    ;

exp:
    NUM
    | exp exp '+'
    | exp exp '-'
    | exp exp '*'
    | exp exp '/'
    | exp exp '^'
    ;

%%

```

```
void yyerror(const char *s) {  
    printf("Invalid postfix expression\n");  
    exit(1);  
}  
  
int main() {  
    printf("Enter postfix expression: ");  
    yyparse();  
    return 0;  
}
```

Output

```
CD_A1@CL3-25:~/Desktop/College/Sem6/CD/Lab/Lab10$ ./q4  
Enter postfix expression: 10 2 8 * + 3 -  
Valid postfix expression
```